

Apprentissage par renforcement

Appliqué au jeu vidéo



Programme

- Séance 1
 - Principe de l'apprentissage par renforcement
 - Exemples d'applications
 - Modélisation de problèmes pour l'apprentissage par renforcement
 - Mise en place du projet (contraintes, barème, etc.)
- Séance 2
 - **Groupes constitués avec propositions de sujets**
 - *Live coding* :
 - Modélisation d'un jeu de plateau
 - Implémentation d'un algorithme TD-Learning
 - Suivi projet : validation du sujet
 - Pédagogie sur les sujets choisis et les modélisations possibles
- Séance 3
 - *Live coding* :
 - - interface graphique du jeu
 - Paramétrage de l'algorithme, visualisation des résultats
 - **Suivi projet : modélisation complète (présentation) avec implémentation**
- Séance 4
 - **Contrôle continu : modélisation en apprentissage par renforcement**
 - Techniques de modélisation par radar
 - Compromis exploration / exploitation
 - **Suivi projet : implémentation apprentissage**
- Séance 5
 - Restitution contrôle continu
 - Réalisation de jeux avancés : Deep Reinforcement Learning, multi-joueurs (double Q-Learning), etc.
 - Approches complémentaires (algorithmes évolutionnaires pour le jeu, etc.)
 - **Suivi projet : présentation du prototype (apprentissage + IHM)**

Barème projet



Présentation	Compréhension algorithme	Qualité du code	IHM
Informations à l'écran	Graphique / Evaluation	Modélisation / Paramétrage	Amélioration du score
Sauvegarde de l'apprentissage	Qualité Qtable / RN	Exploration	Learn mode / Play mode



Principes de l'apprentissage par renforcement

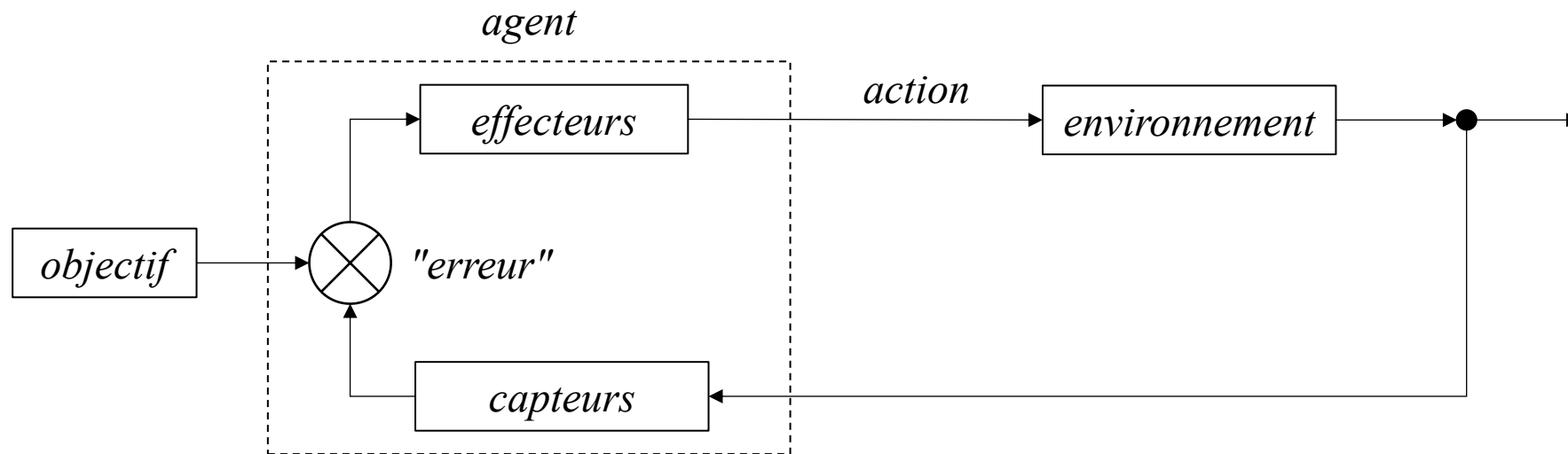


Rappel sur les différents types d'apprentissage

- Apprentissage supervisé
 - Régression
- Apprentissage non-supervisé
 - Clustering
- Apprentissage par renforcement
 - Apprentissage de décisions
 - Par exploration, essais et erreurs
 - En interaction avec un environnement partiellement inconnu



Rétroaction



Agent et environnement

- Agent

- Programme « intelligent »
- Programme apprenant
 - Pas « évoluant »
- Robot

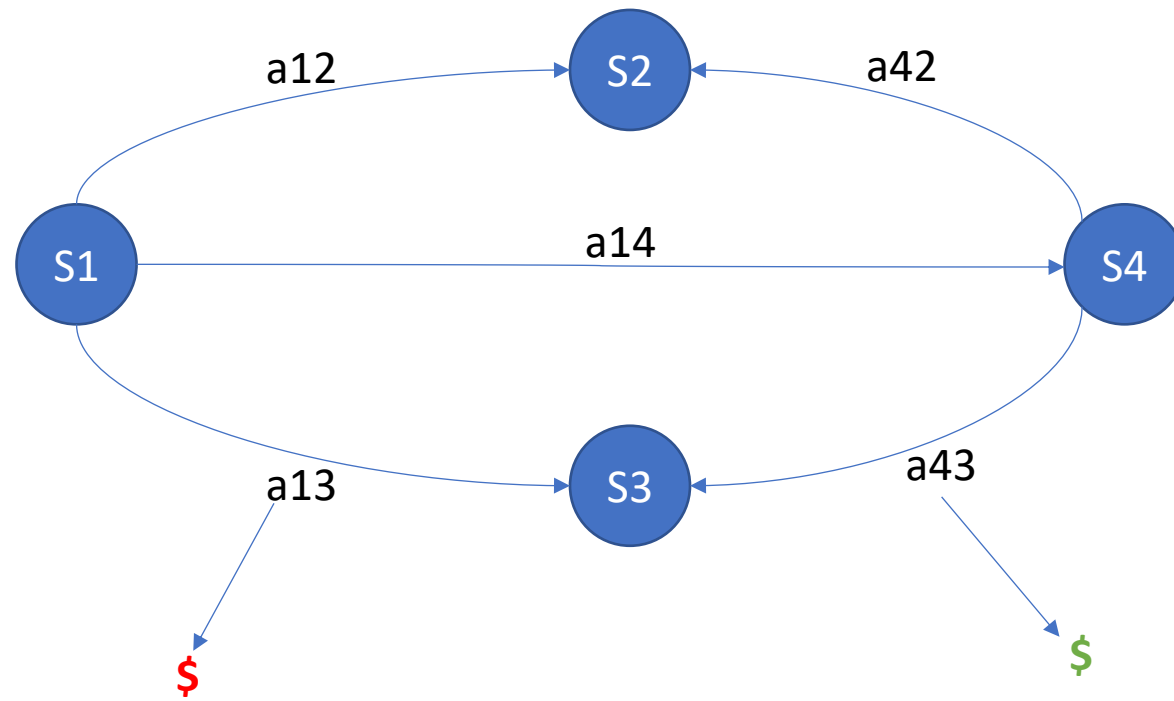
- Environnement

- "Réalité" sous jacente à des données captées en réponse à une action
- "Réel ou virtuel"

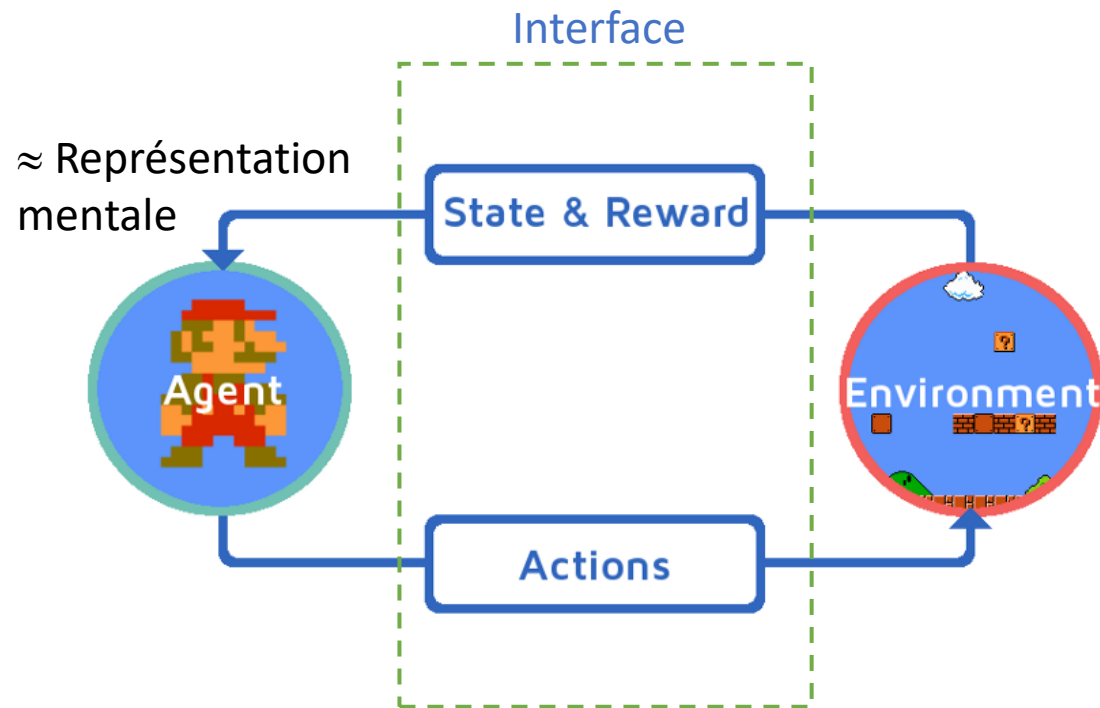
Processus de décision markovien (*MDP*)

- Apprentissage par renforcement : méthode permettant de résoudre des processus de décision markoviens
- Fonction de transition
 - $T(s, a, s') = p(s'|s, a) \in [0,1]$
 - s est l'état du processus
 - a est l'action de l'agent
 - s' est l'état du processus après avoir appliqué l'action a
- Récompense associée
 - $R(s, a, s')$
- Cas déterministe
 - $s' = T(s, a)$

Principe



Principe



- États
 - S
- Actions
 - A
- Rétribution (a posteriori)
 - *Delayed reward*
 - $R > 0$: récompense
 - $R < 0$: réprimande / punition
 - $R = 0$
- $S_0 \rightarrow A_0 \rightarrow \{S_1, R_1\} \rightarrow A_1 \rightarrow \dots$
- Transitions
 - $S_0 \rightarrow S_1 \rightarrow S_2 \rightarrow \dots$
- Objectif
 - Maximiser la récompense totale
 - Critère d'arrêt
 - Convergence

Politique π

Politique

Règle de décision : π_t détermine, à l'instant t , une loi d'action en tout état :

– déterministe : $\pi_t : X \rightarrow A$,

$\pi_t(x)$ = action choisie en x ,

– stochastique : $\pi_t : X \times A \rightarrow \mathbb{R}$,

$\pi_t(a|x)$ = probabilité de choisir a en x .

Politique (ou **stratégie**, ou **plan**) : $\pi = (\pi_0, \pi_1, \pi_2, \dots)$

Politique *stationnaire* ou *Markovienne* : $\pi = (\pi, \pi, \pi, \dots)$.

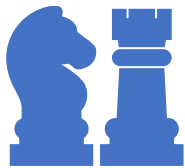
→ Pour une politique $\pi = (\pi_0, \pi_1, \dots)$ donnée, la séquence d'états $(x_t)_{t \geq 0}$ devient une chaîne de Markov de probabilités de transition :

$p(y|x) = p(y|x, \pi_t(x))$.

Environnement

- Déterministe vs. stochastique
 - Lancé de dé, Tétris, etc.
- Hostile ou non
 - Chess vs. Tétris
- Partiellement observable

Classification des jeux



Déterminés

Information complète

- Stratégie combinatoire abstraite
- Échecs, dames

Information incomplète

- Stratego, Diplomacy, Mastermind



Mixtes

Information complète

- Petits chevaux

Information incomplète

- Poker, Scrabble, Cluedo



Pur hasard

Roulette, Bataille

Qu'est-ce que RL peut apprendre ?

- Apprentissage d'une stratégie comportementale (politique π) pour la prise de décision en milieu incertain, à partir de récompenses ou pénalités
- Tables de correspondance
- Fonctions d'évaluation
 - Machine Learning
- Hypothèses
 - Hypothèse déterministique ou stochastique
 - Hypothèse markovienne

Différentes méthodes

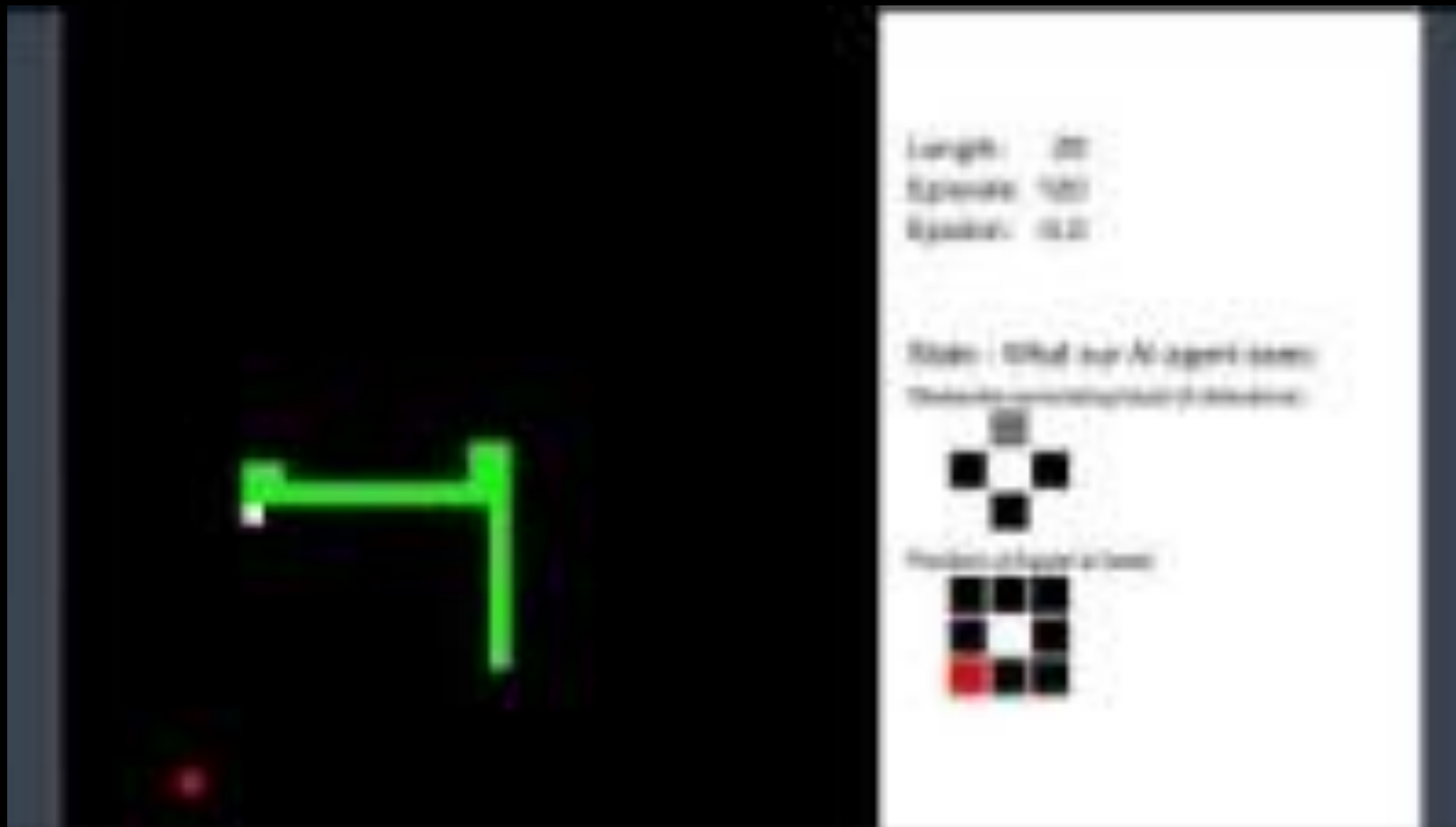
- Richard Sutton
 - *Temporal Credit Assignment in Reinforcement Learning*, 1984
- TD-Learning (coefficient Lambda)
 - *Temporal Differences*
 - *Model free*
 - Q-Learning
 - Chris Watkins, 1989
 - SARSA
 - *State-Action-Reward-State-Action*
 - Rummery and Niranjan, 1994
 - Problème de dimension
- Actor-Critic
- R-Learning
- Deep Reinforcement Learning
 - End-to-end RL : apprentissage neuronal pur
 - TD-Gammon 1992 $TD(\lambda)$
 - DQN (CNN)
 - Deep Q-networks



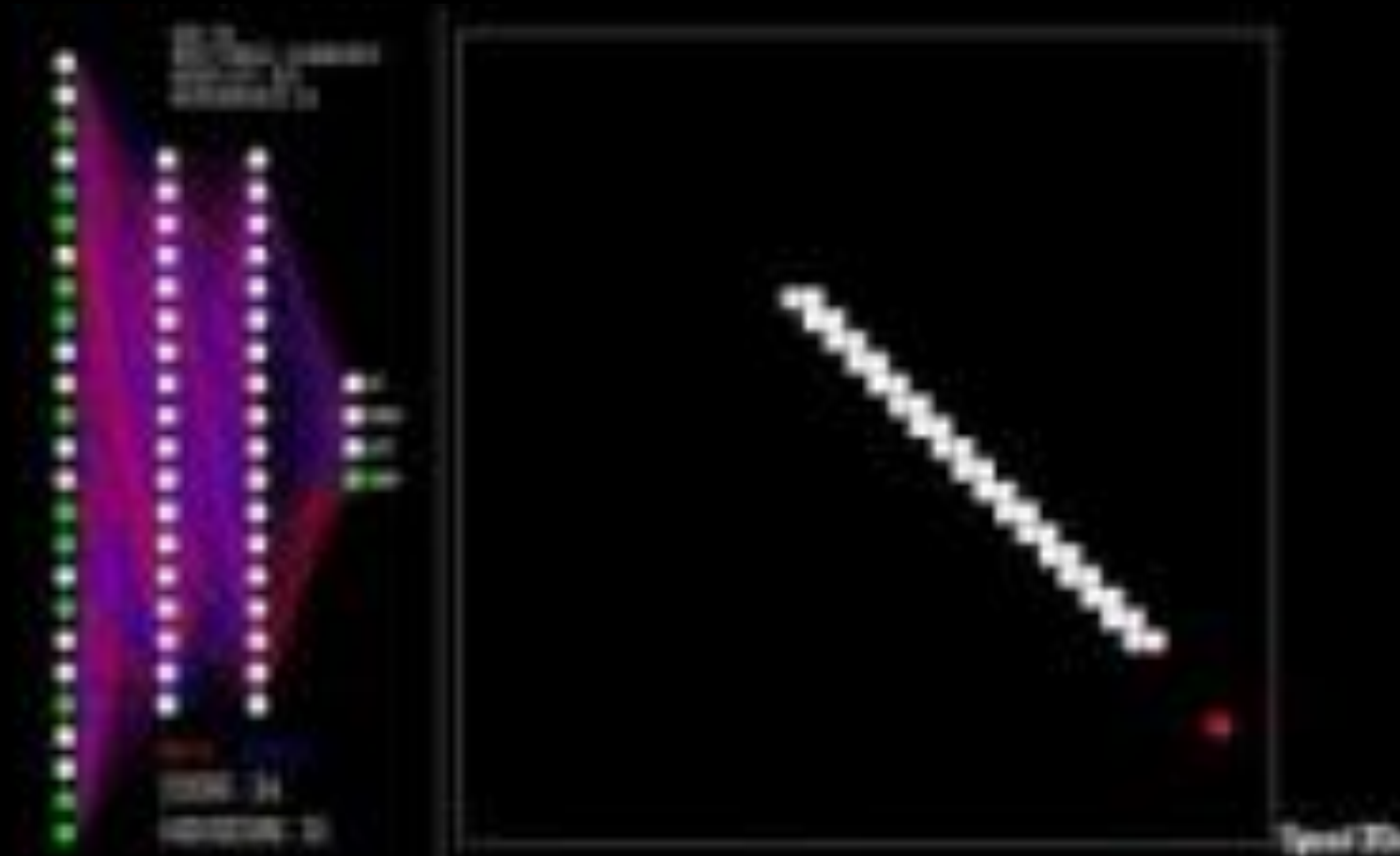
Examples



Snake



Snake (version RN)



Super Mario



Starcraft



Grand Turismo Sport

Our Approach



TG



Parking



Hide and Seek (multi-agents)



Simulation physique



Robotique



Approches similaires

Exemples

Super Mario (algorithmes génétiques)



Autre approche : algorithmes génétiques



Différence avec l'apprentissage supervisé

- Experience vs. « Teacher signal »
- Pas de dataset
- Online

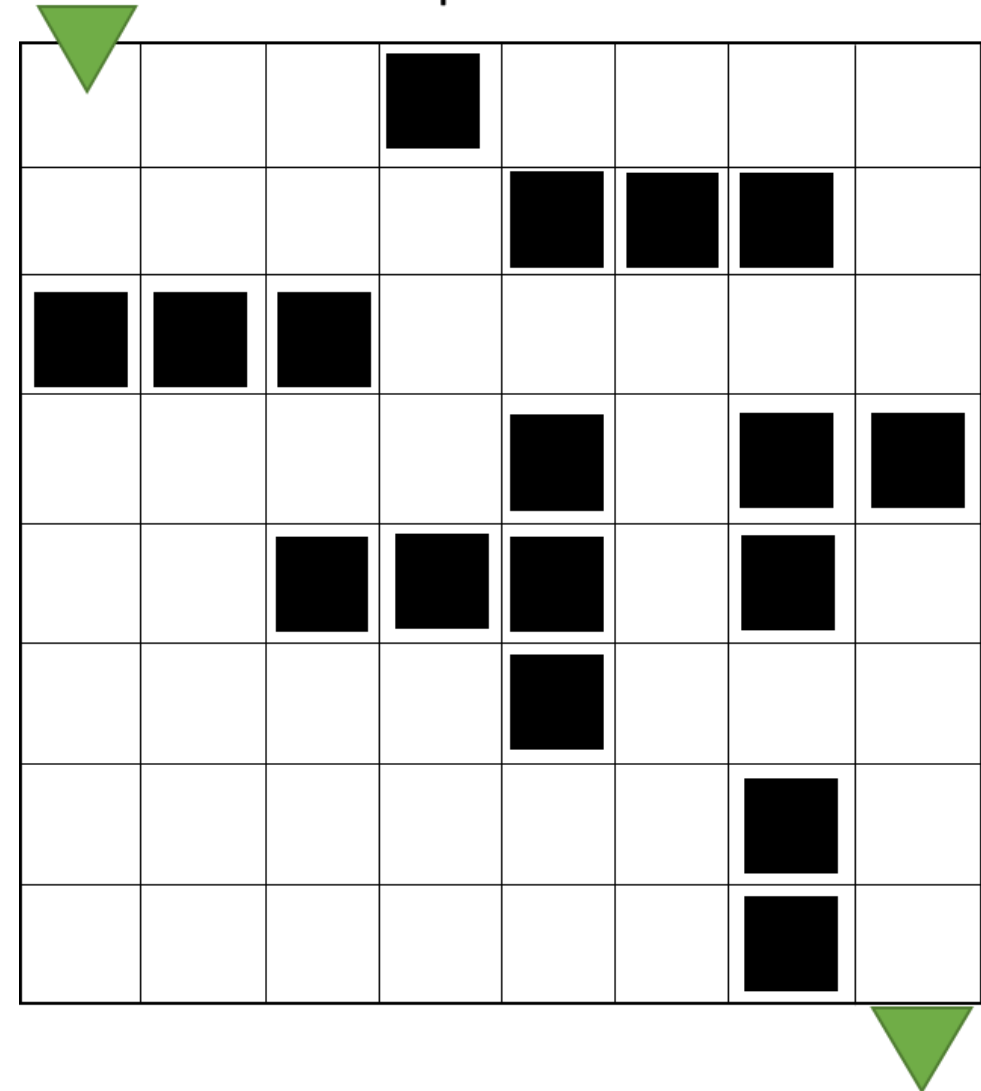
Modéliser un jeu

L'exemple du labyrinthe



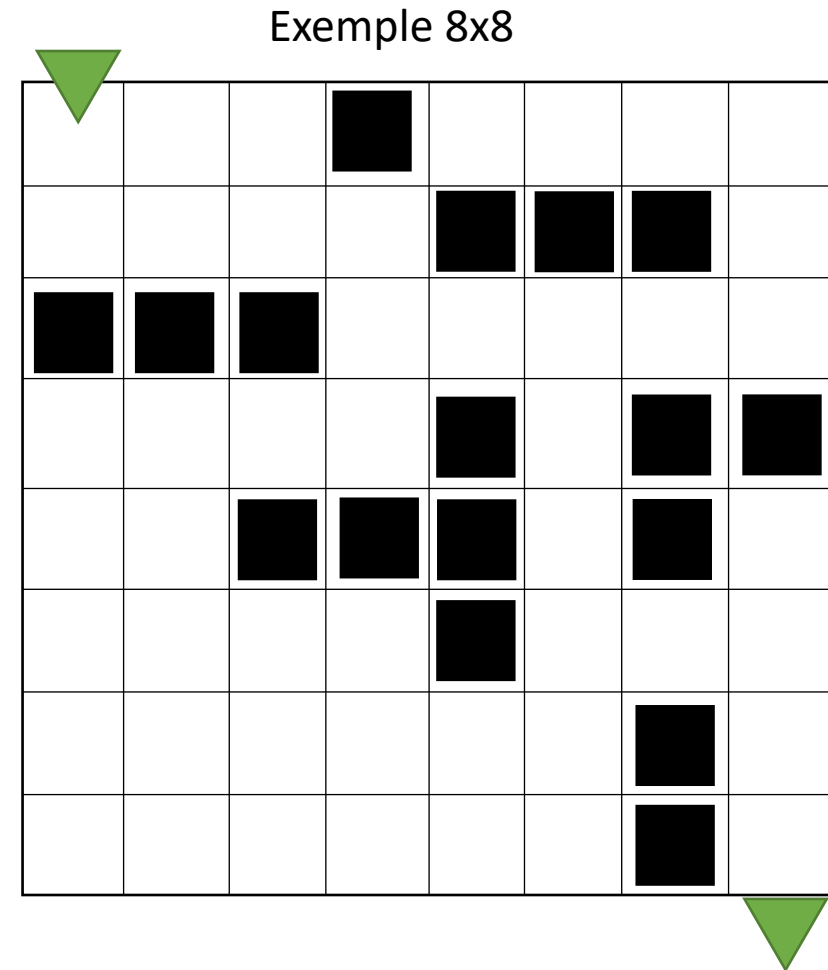
- 

Exemple 8x8

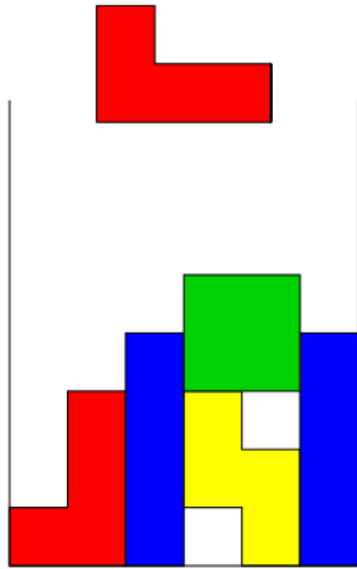


Modélisation complète

- Environnement
 - 64 états
 - (0, 0)... (7, 7)
 - (ligne, colonne)
- Actions
 - U, D, L, R
- Récompenses
 - Mur : -128
 - Case vide : -1
 - Sortie : 64



Tétris



- Etat: configuration du mur + nouvelle pièce
- Action: positions possibles de la nouvelle pièce sur le mur,
- Récompense: nombre de lignes supprimées
- Etat suivant: nouvelle configuration du mur + aléa sur la nouvelle pièce.

Il est prouvé que pour toute stratégie, le jeu se finit avec probabilité 1. Donc l'espérance de la somme des récompenses à venir est finie.

Difficulté de ce problème: espace d'états très grand (ex: 10^{61} pour hauteur 20, largeur 10, et 7 pièces différentes).



JOURNAL ARTICLE

How to Lose at Tetris

Heidi Burgiel

The Mathematical Gazette
Vol. 81, No. 491 (Jul., 1997), pp. 194-200
(7 pages)
Published By: The Mathematical
Association

This concludes the proof of Theorem 1: in a Tetris game in which only Z-tetrominoes of alternating orientation appear, any player will lose after placing at most 69,600 tetrominoes.

Comment généraliser ?



Nous avons appris que
cette situation est
mauvaise



Pb : cela ne nous
renseigne pas sur celle-ci



Ni même sur celle-là !

Installer Matplotlib

- `pip install matplotlib`

Algorithmes de base

Politique de mise à jour (*policy update*)

- Méthode essai et erreur
 - *Trial and error*
 - Méthode scientifique itérative
 - Dans le cadre informatique de la résolution de problèmes :
 - *Generate and test*
 - Jeux vidéos : *die and retry*
- Prise de décision
 - $\{p(A_i|S_j)\}_{i,j}$
- Maximiser le cumul des récompenses
 - $R_t = r_t + \gamma R_{t-1}$
 - $r_t + \gamma r_{t-1} + \gamma^2 r_{t-2} + \dots$
- Cumul des récompenses

Q-table

		Actions				
États	(0, 0)	R(a s)	U	D	L	R
	(0, 1)					
	.					
	.					
	(N, P)					

Q-Learning

- $Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right]$
- α : taux d'apprentissage (*learning rate*)
- γ : taux de diminution des renforcements (*discount factor*)
 - Facteur d'actualisation, coefficient d'actualisation, etc.

Mise à jour de la politique

- Problème déterministe
 - $\alpha = 1$
- Problème stochastique
 - α fixe < 1
 - Ou réduire progressivement pour converger

Choix d'alpha et gamma

- Alpha [0 ; 1]
 - Vitesse d'apprentissage
 - Prise en compte de la récompense instantanée
 - Attention en environnement stochastique : la récompense est variable
 - $\text{Alpha} < 1$
 - Environnement déterministe (sûr) : $\text{alpha} = 1$
- Gamma [0 ; 1]
 - Prise en compte de la temporalité
 - Si gamma trop faible, risque de minimum local (boucles, etc.)
 - Si gamma trop fort en environnement stochastique, risque de prise en compte de récompenses aléatoires

Variante : SARSA

- $Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]$
- α : taux d'apprentissage (*learning rate*)
- γ : taux de diminution des renforcements (*discount factor*)
 - Parfois "facteur d'actualisation"

Double Q-Learning

Algorithm 1 Double Q-learning

```
1: Initialize  $Q^A, Q^B, s$ 
2: repeat
3:   Choose  $a$ , based on  $Q^A(s, \cdot)$  and  $Q^B(s, \cdot)$ , observe  $r, s'$ 
4:   Choose (e.g. random) either UPDATE(A) or UPDATE(B)
5:   if UPDATE(A) then
6:     Define  $a^* = \arg \max_a Q^A(s', a)$ 
7:      $Q^A(s, a) \leftarrow Q^A(s, a) + \alpha(s, a) (r + \gamma Q^B(s', a^*) - Q^A(s, a))$ 
8:   else if UPDATE(B) then
9:     Define  $b^* = \arg \max_a Q^B(s', a)$ 
10:     $Q^B(s, a) \leftarrow Q^B(s, a) + \alpha(s, a) (r + \gamma Q^A(s', b^*) - Q^B(s, a))$ 
11:  end if
12:   $s \leftarrow s'$ 
13: until end
```

Enregistrer
et recharger
un modèle

```
import pickle

filename = "clf.pkl"

with open(filename, 'wb') as file:
    pickle.dump(clf, file)

with open(filename, 'rb') as file:
    clf = pickle.load(file)
```

Arcade

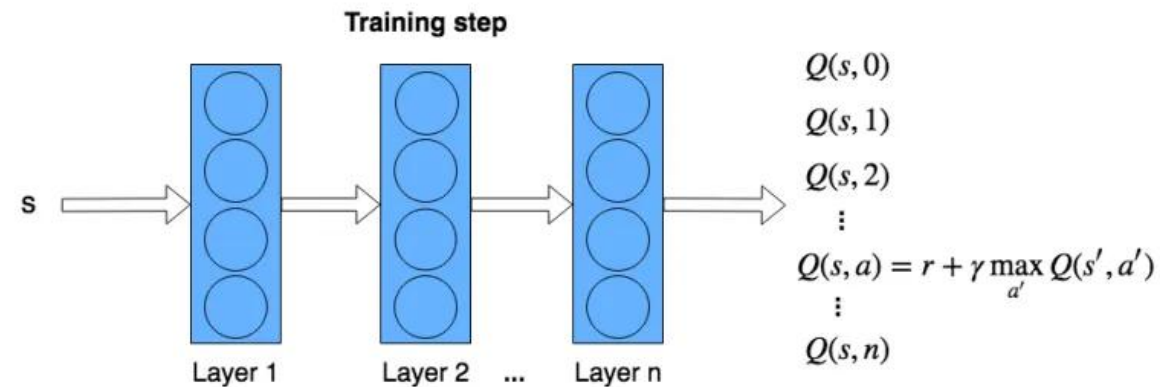
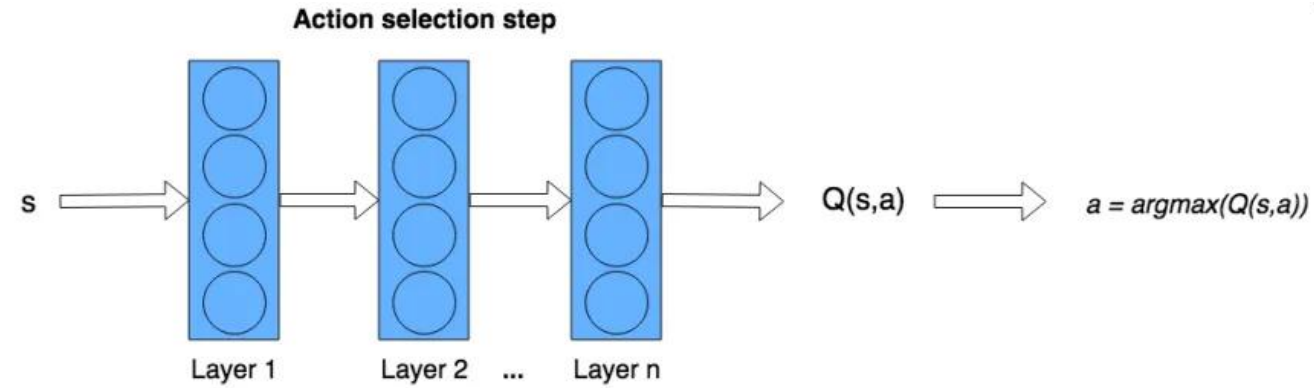
Librairie Python



Installer Arcade

- `pip install arcade`
- ~~Optionnel : `pip install dataclasses`~~
- <https://api.arcade.academy/>
- Exemples
 - <https://api.arcade.academy/en/latest/examples/>

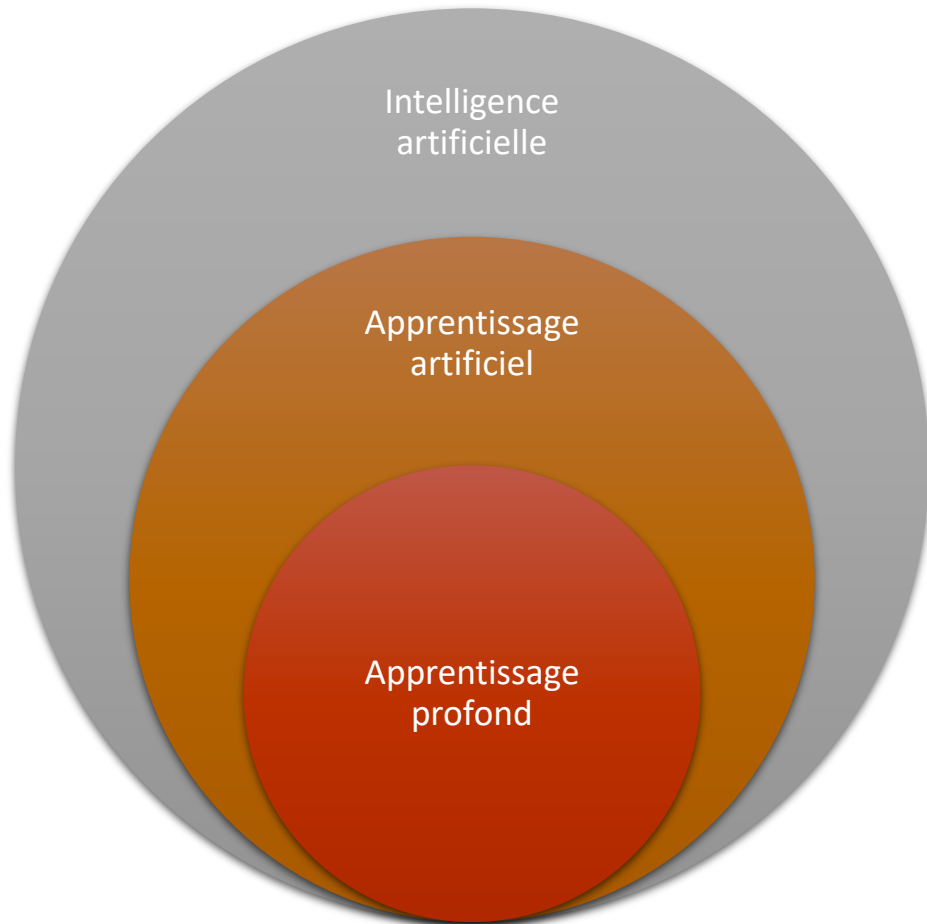
Deep-Q Learning



- Utile quand l'espace d'états devient trop important
- Jeu de Go

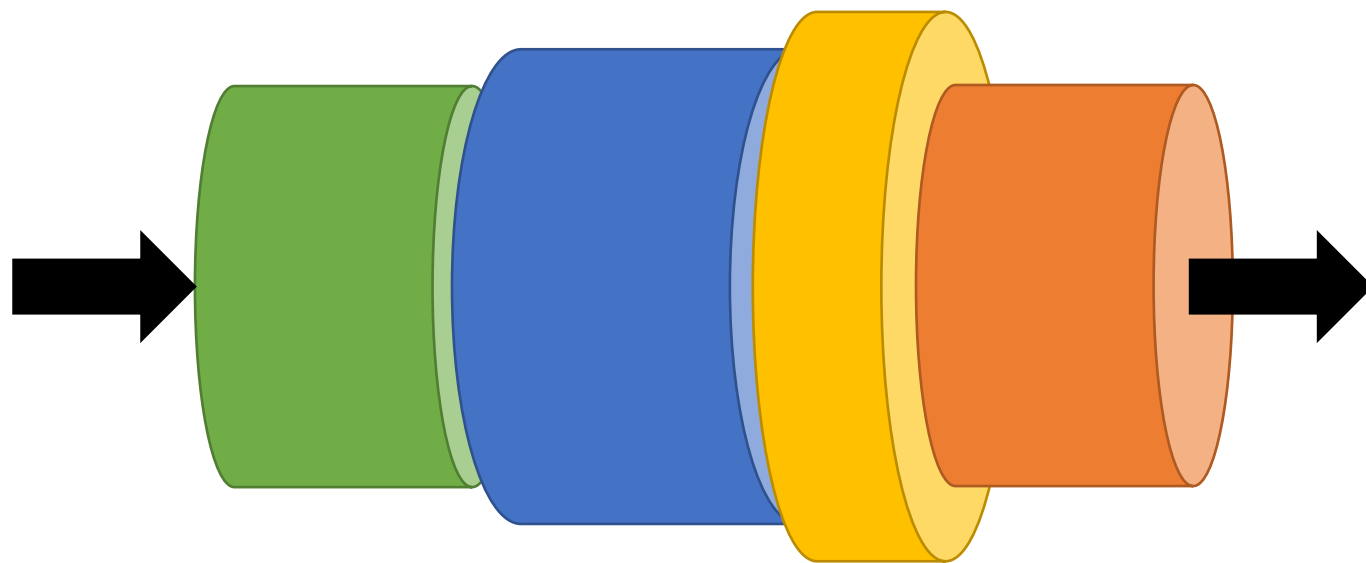
Introduction aux réseaux de neurones

Deep learning



- Imiter et étudier les fonctions cognitives humaines
- Apprendre sans programmation explicite
- Apprendre à partir de données avec des architectures connexionnistes de type « réseaux de neurones »

Architectures profondes

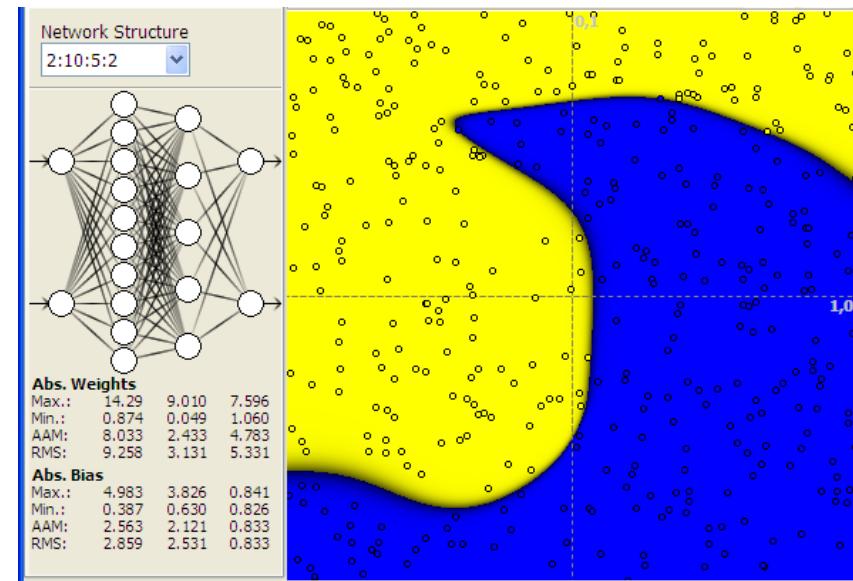


Exemples de tâches

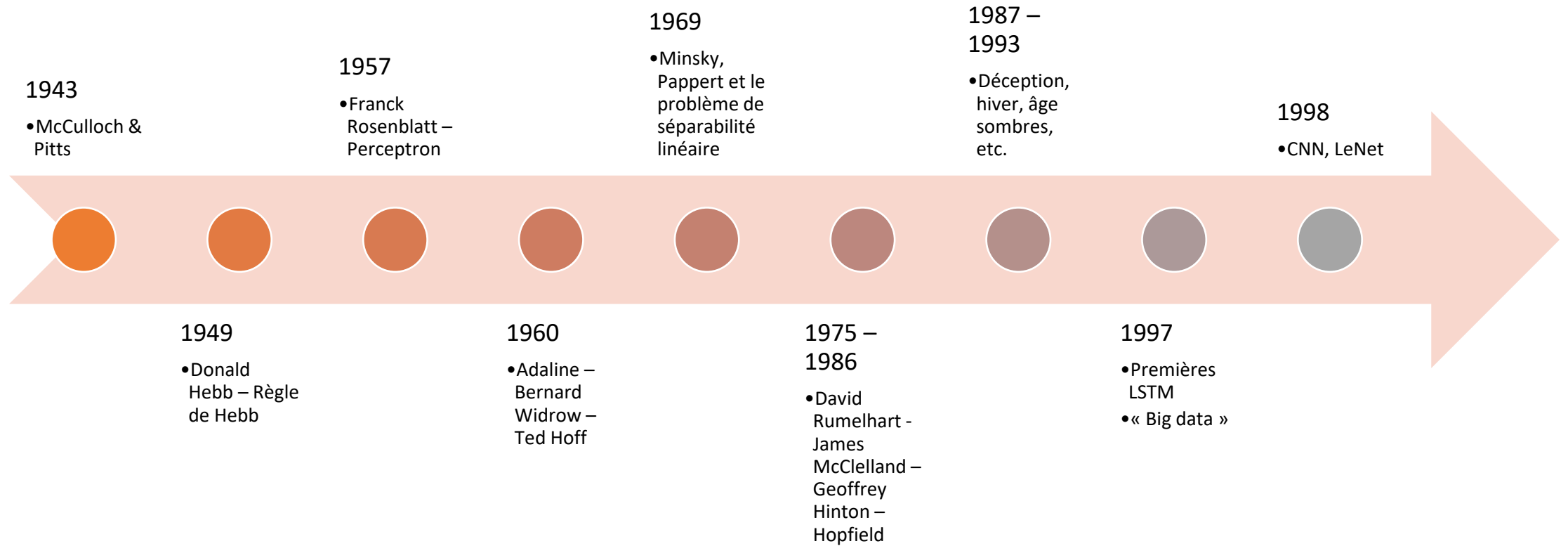
- Classification (détection)
 - Détection de fraude, *credit scoring*,
 - Images
 - Visages, galaxies, diagnostic médical, reconnaissance de texte
 - Vidéo
 - Véhicule autonome
 - Son
 - Voix : *Siri*, *Cortana* et *Google Now*
 - Texte
 - Traduction
- Régression (estimation, prévision)
 - Modélisation scientifique
 - Création multimédia : visuelle, sonore, textuelle
 - *Inpainting*, *deep fake*, réparation d'image
 - Génération séquentielle
 - Jeux
- Auto-encodage
 - Débruitage
 - Création : visuelle, sonore, etc.

Régression et classification

- Régression
 - $f: \mathbb{R}^n \rightarrow \mathbb{R}^p$
- Classification
 - $f: \mathbb{R}^n \rightarrow \mathbb{N}^p$
 - La classe s'obtient généralement par réduction d'un vecteur probabiliste
 - *Max, softmax, etc.*

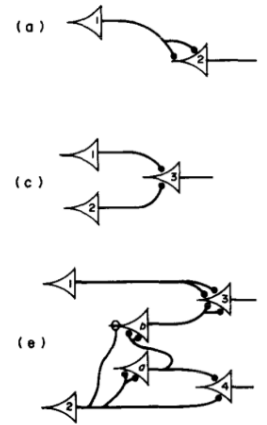
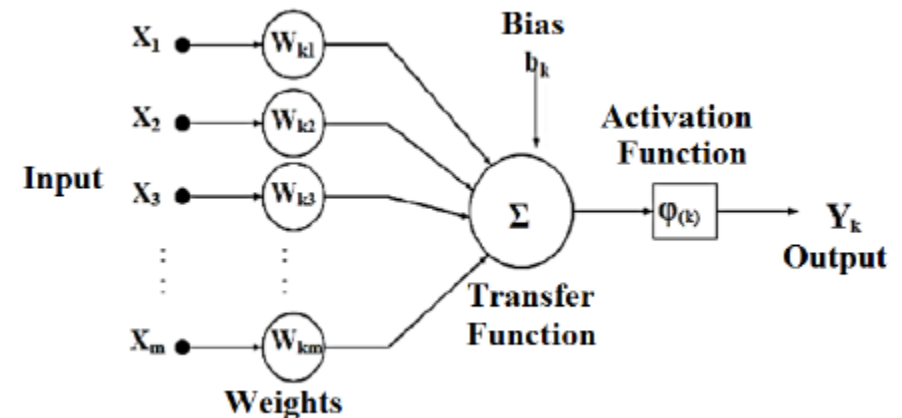


Histoire du connexionnisme



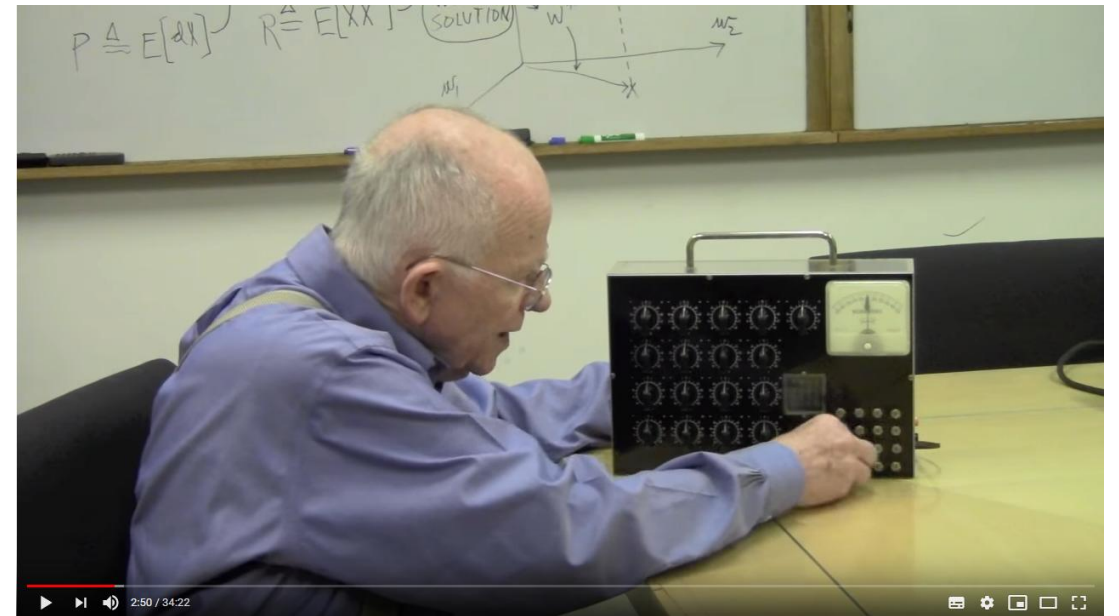
Neurone formel

- Warren McCulloch , neurologue et Walter Pitts, psychologue, 1943
- $y(x_1, \dots x_n) = \varphi(w_0 + \sum_{i=0}^n w_i x_i)$
- Sous forme matricielle
 - $Y(X) = \varphi(X^T W)$



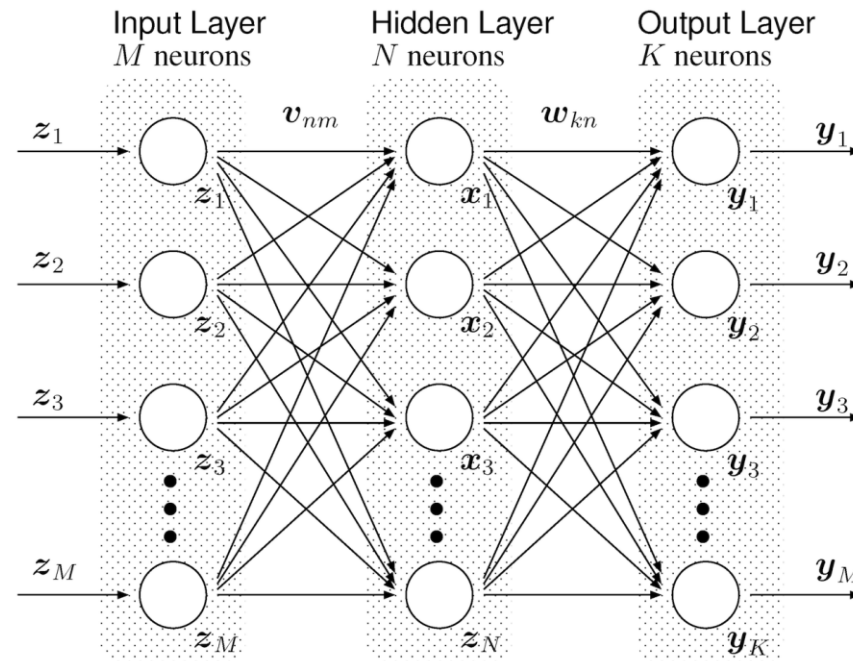
Adaline

- Bernard Widrow et Ted Hoff, ingénieurs, 1960
- $w_{i,j} \leftarrow w_{i,j} + \underbrace{\eta \cdot (d_j - o_j)}_{\Delta w_{i,j}} \cdot x_i$

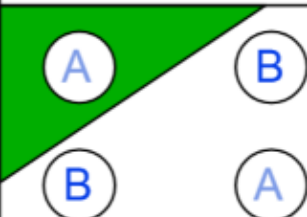
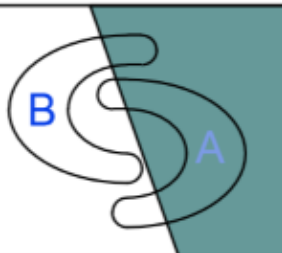
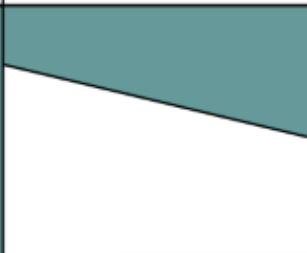
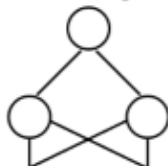
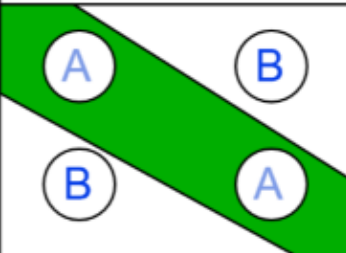
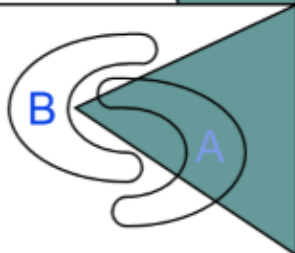
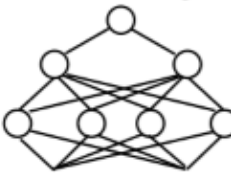
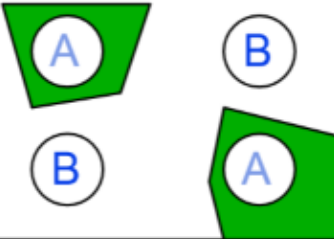


Perceptron multicouches

- Paul Werbos, sociologue, 1974 et David Rumelhart et James McClelland, psychologues, 1984

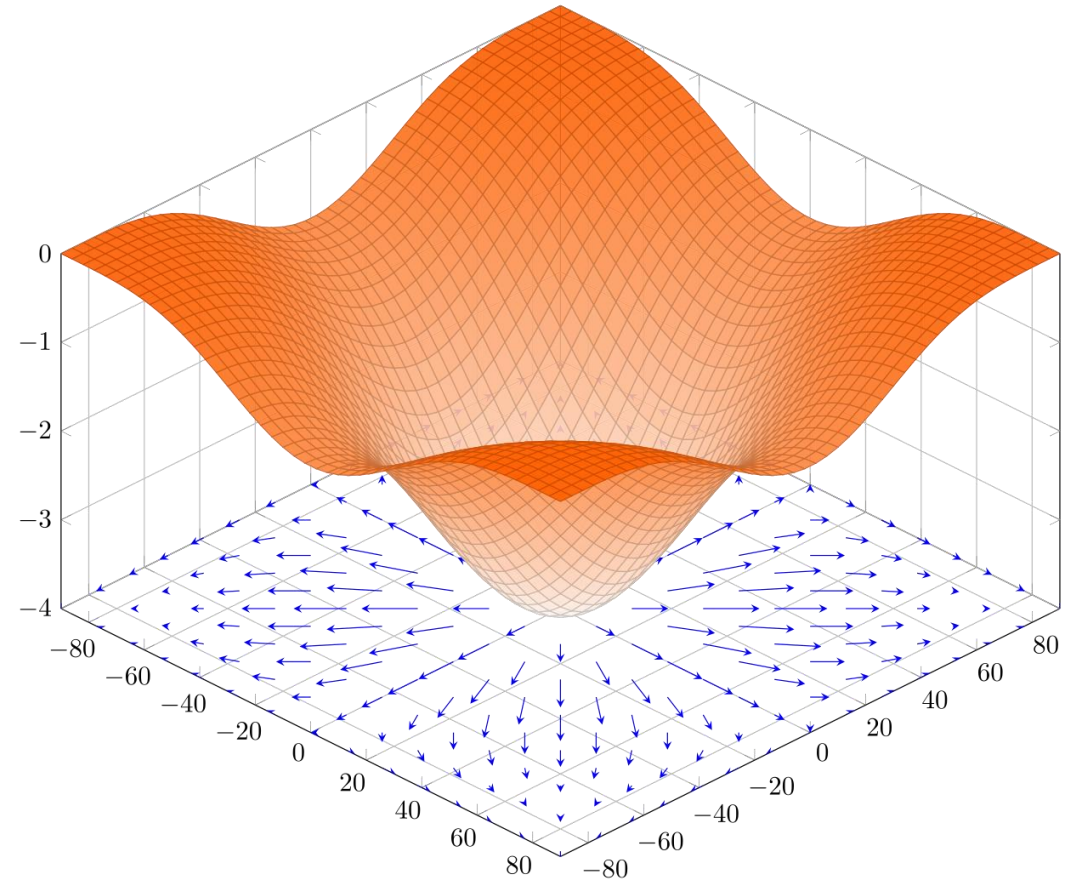


Ce que peut apprendre un MLP

Structure	Types of Decision Regions	Exclusive-OR Problem	Classes with Meshed regions	Most General Region Shapes
Single-Layer 	Half Plane Bounded By Hyperplane			
Two-Layer 	Convex Open Or Closed Regions			
Three-Layer 	Arbitrary (Complexity Limited by No. of Nodes)			

Rétropropagation du gradient

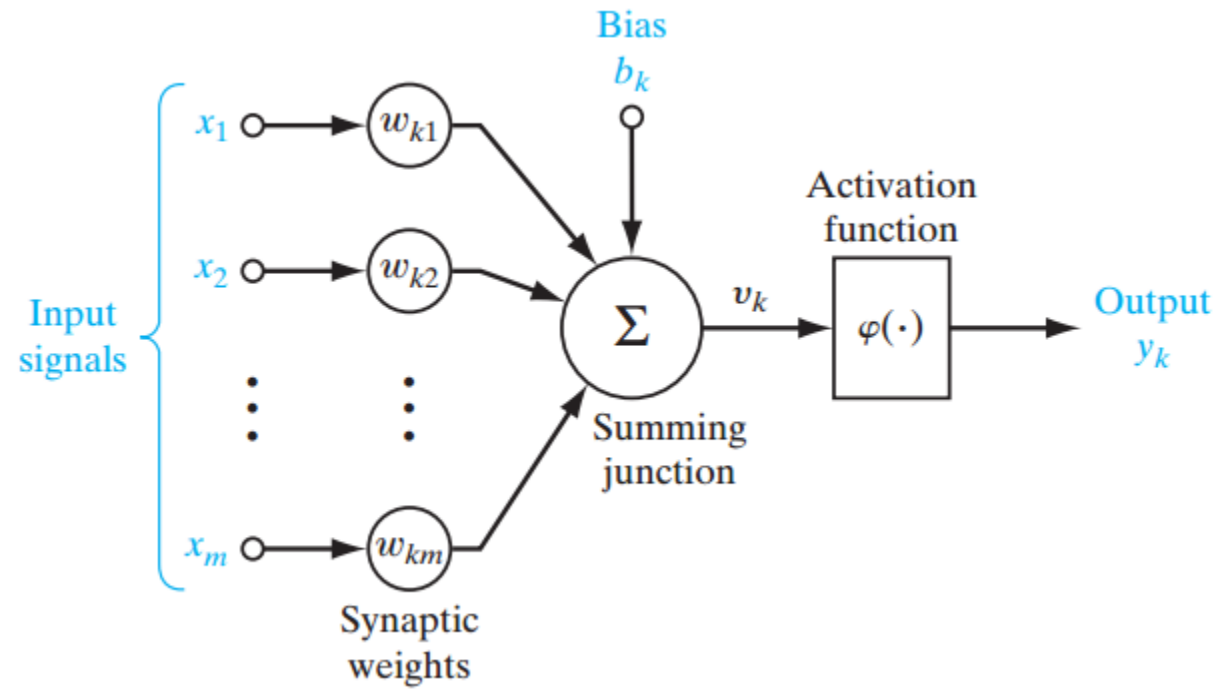
- $\Delta W_{ij} = \eta \cdot \delta_j \cdot Y_i$
- $\delta_j = (T_j - Y_j) \cdot f'_j(\sum_i W_{i,j} Y_i)$
- $\delta_j = \sum_k \delta_k W_{k,j} \cdot f'_j(\sum_i W_{i,j} Y_i)$



Prévenir le surapprentissage

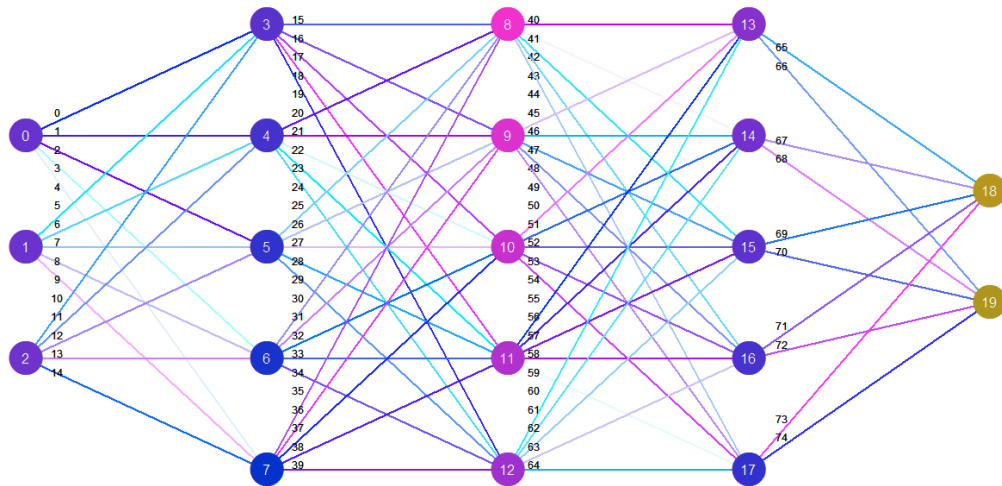
- Limiter la complexité du MLP
 - Nombre de couches cachées
 - Nombre de neurones sur chaque couche
- Régularisation
 - Pénaliser les poids élevés
- Interrompre l'apprentissage
 - *Early stopping*

Poids et biais

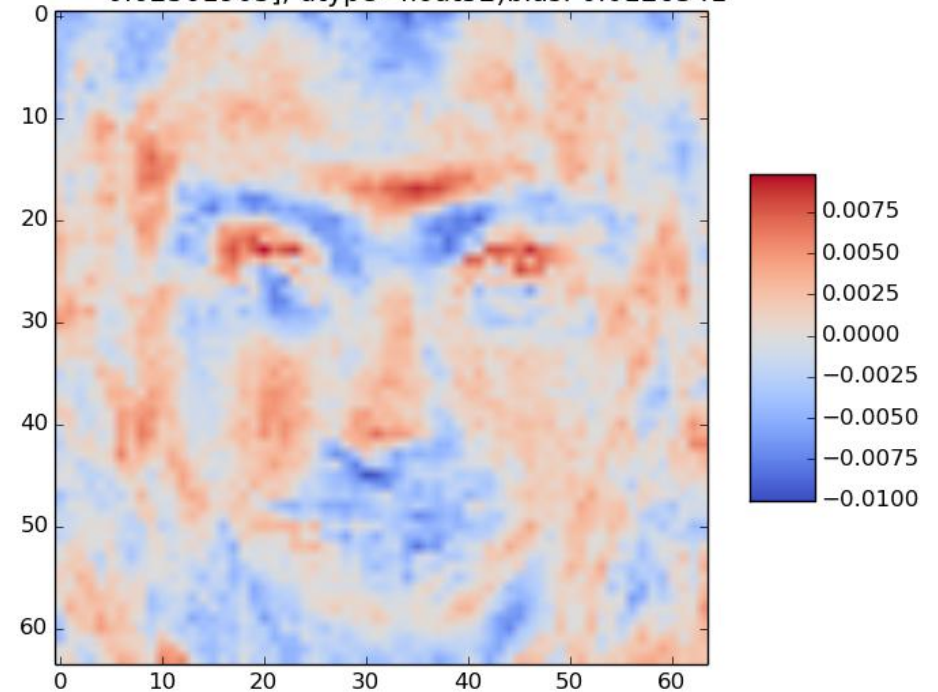


Visualiser les poids

Analogie avec les champs réceptifs



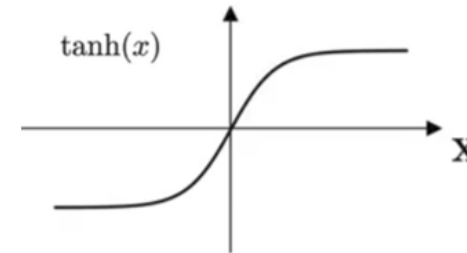
s: array([-0.0660211 , -0.02434859, -0.10672989, 0.00908299, 0.08226717,
0.02301903], dtype=float32) bias: 0.0126341



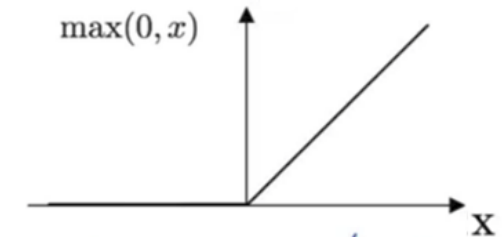
Fonctions d'activation

- Identité
 - Régression linéaire
 - Réseau de neurones équivalent à un simple produit matriciel
- Fonctions non linéaires
 - ReLU
 - $f(x) = \max(0, x)$
 - *Rectified Linear Unit*
 - *Leaky ReLU*
 - $f(x) = \max(\epsilon x, x)$
 - Sigmoide
 - $f(x) = \frac{1}{1+e^{-x}}$
 - Tangente hyperbolique
 - $f(x) = \tanh(x)$

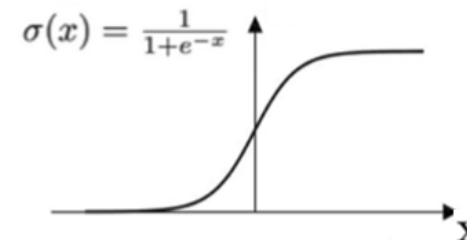
Hyper Tangent Function



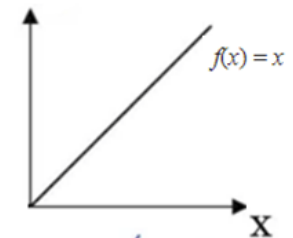
ReLU Function



Sigmoid Function



Identity Function

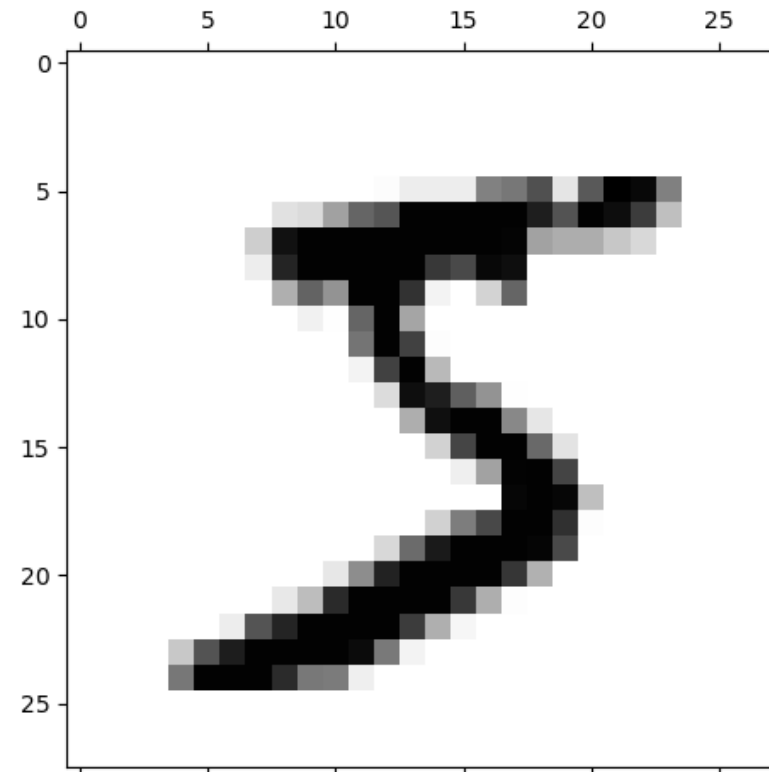


Perceptron multi-couches (*MLP*)

- Pourquoi la théorie ?
 - Parce coder-exécuter devient impossible ?
 - Donc, il faut coder bien d'abord !
 - Adieu aux scripteurs fous...
- TP avec scikit-learn et openml.org
- <https://www.openml.org/d/554>

Données d'entrée

- Données de test
- Données d'apprentissage
- Préparation
- Visualisation

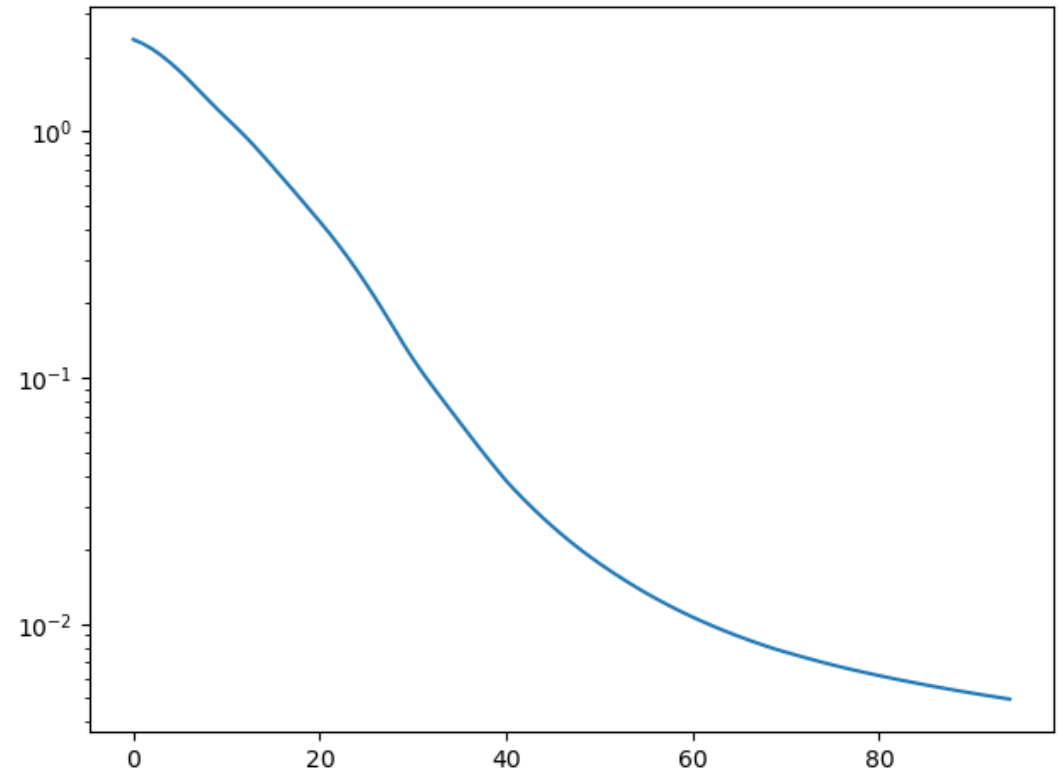


Apprentissage

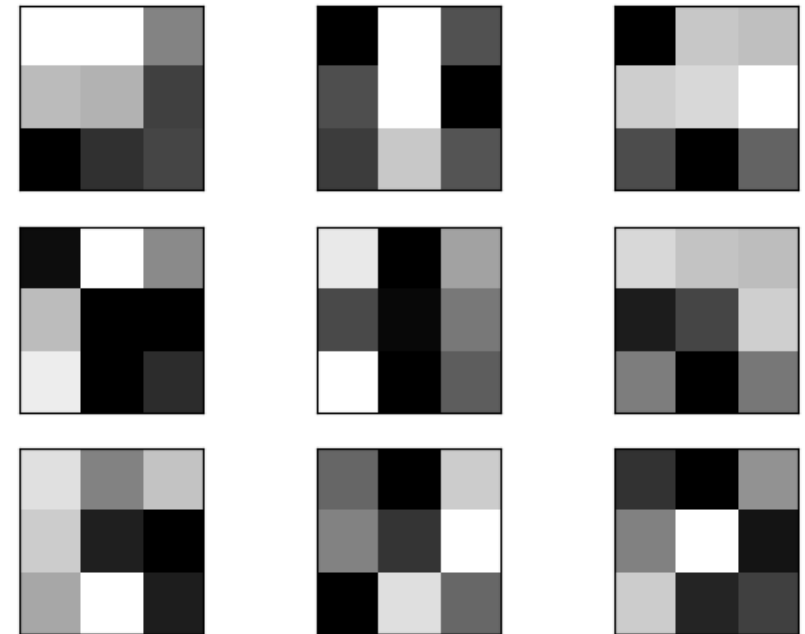
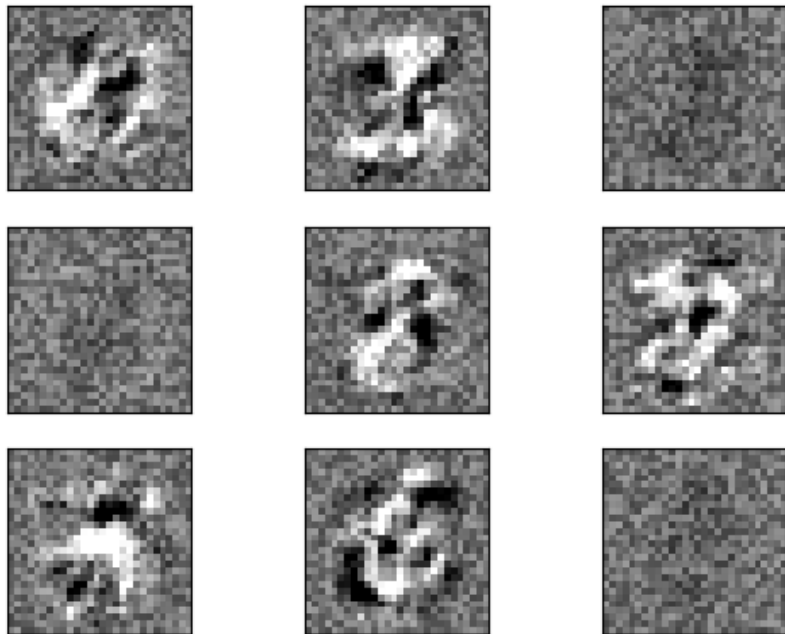
- (Hyper-)paramètres

```
class sklearn.ensemble.GradientBoostingClassifier(  
    loss='deviance',  
    learning_rate=0.1,  
    n_estimators=100,  
    subsample=1.0,  
    criterion='friedman_mse',  
    min_samples_split=2,  
    min_samples_leaf=1,  
    min_weight_fraction_leaf=0.0,  
    max_depth=3,  
    min_impurity_decrease=0.0,  
    min_impurity_split=None,  
    init=None,  
    random_state=None,  
    max_features=None,  
    verbose=0,  
    max_leaf_nodes=None,  
    warm_start=False,  
    presort='auto')  

```



Visualisation des poids



Exercices

- 1 neurone
- MLP
- Framework *from scratch*

Réserve

Exploration

- Aléatoire + Decay

Frameworks

- OpenAI
- Keras
- Python
- Scikit-learn

Autres approches

- Apprentissage génétique
 - Mar I/O : <https://youtu.be/qv6UV0Q0F44>
 - Circuits de voitures : <https://youtu.be/Aut32pR5PQA>

